Submitted By:

Muhammad Hadi Shahid - 2026-CYS-036

Submitted To:

Sir Hafiz Muhammad Mubashir

For the fulfillment of,

Programming Fundamentals Lab

Department of Computer Engineering

University of Engineering and Technology, Lahore

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Mannual Report	**Dated:** 28-06-2026
Semester: 1st Semester	**Session:** 2026 Spring
Lab/Project/Assignment #: 1	**CLOs to be covered:** CLO 1
Lab Title: Input/Output, Data Types, Operators, Conditional Statements (If/Else)	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No. 1 :-

Lab Objectives:

To understand and apply the fundamental concepts of Python programming including Input/Output operations, Data Types, Operators, and Conditional Statements (If/Else).

Equipment Required:

To perform the following tasks we require:

- Python (Installed in the computer)
- An IED (VS code, Pycharm, etc)

Theory:

Python supports various **Data Types** such as int, float, string, and boolean. **Input/Output** is handled using input() and print() functions. **Operators** include arithmetic, comparison, and logical operators. **Conditional Statements** like if/else allow decision-making in a program based on given conditions.

Lab Work:

Task # 1:

```
a = 254

b = 610

c = a / b  * 100

print ("Your average is : ",c )
```

Result:

```
Your average is :  41.63934426229508
```

Task # 2:

```
print ("Hello, World!")
```

Result:

```
Hello, World!
```

Task # 3:

```
a = input("Enter 1st value : ")
b = input("Enter 2nd value : ")
c = a/b  * 100
print ("Your average is : ",c )
```

Result:

```
Enter 1st value : 254
Enter 2nd value : 610
Your average is :  41.63934426229508
```

Task # 4:

```
c = a/b * 100
print ("Your average is : ", c)
if c == 0:
    print ("Invalid Grade")
elif c > 100:
    print ("Invalid Grade")
elif a > b or a > 300:
```

```
    print ("Invalid Grade")
else:
    if c >= 90:
        print ("Grade A")
    elif c >= 85:
        print ("Grade A-")
    elif c >= 80:
        print ("Grade B+")
```

-
-
-

Result:

```
Enter Obtained Marks : 75
Enter Total Marks : 100
Your average is :  75.0
Grade B
```

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Mannual Report	**Dated:** 28-06-2026
Semester: 1st Semester	**Session:** 2026 Spring
Lab/Project/Assignment #: 1	**CLOs to be covered:** CLO 1
Lab Title: For Loop	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No. 2 :-

Lab Objectives:

To understand and implement For Loops in Python, and to practice iterating over sequences, ranges, and collections using loop structures.

Equipment Required:

To perform the following tasks we require:

• Python (Installed in the computer)
• An IED (VS code, Pycharm, etc)

Theory:

A **For Loop** in Python is used to iterate over a sequence (such as a list, tuple, string, or range). It executes a block of code repeatedly for each item in the sequence. The range() function is commonly used with for loops to generate a sequence of numbers. For loops help in automating repetitive tasks and make code more efficient and readable.

Lab Work:

Task # 1:

```
for i in range(d):
    c = a/b * 100
    print ("Name : ", e)
    print ("Roll NO. : ", f)
    print ("Obtained Marks : ", a)
    print ("Percentage : ", c)
    if c >= 90:
        print ("Grade A")
    elif c >= 85:
        print ("Grade A-")
    elif c >= 80:
        print ("Grade B+")
    elif c >= 75:
        print ("Grade B")
    elif c >= 70:
        print ("Grade B-")
```

-
-
-

Result:

```
Enter total number of students : 2
Enter Student Name : Umar
Enter Student Roll no. : 43
Enter Obtained Marks : 250
Name :  Umar
Roll NO. :  43
Obtained Marks :  250
Percentage :  83.33333333333333
Grade B+
Enter Student Name : Hadi
Enter Student Roll no. : 36
Enter Obtained Marks : 200
Name :  Hadi
Roll NO. :  36
Obtained Marks :  200
Percentage :  66.66666666666667
Grade C+
```

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Mannual Report	**Dated:** 28-06-2026
Semester: 1st Semester	**Session:** 2026 Spring
Lab/Project/Assignment #: 1	**CLOs to be covered:** CLO 1
Lab Title: While Loops	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.

Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No. 3 :-

Lab Objectives:

To understand and implement While Loops in Python, and to practice executing a block of code repeatedly as long as a given condition remains true.

Equipment Required:

- A computer with Python installed
- Any Python IDE (e.g., VS Code, PyCharm, or IDLE)

Theory:

A While Loop in Python repeatedly executes a block of code as long as its condition evaluates to True. Unlike a for loop, the while loop is used when the number of iterations is not known in advance. It requires a loop variable that must be updated inside the loop to avoid an infinite loop. The break and continue statements can also be used to control the flow of a while loop.

Lab Work:

Task # 1:

```
i = 1

while i < d:

   c = a/b * 100

   print ("Name : ", e)

   print ("Roll NO. : ", f)

   print ("Obtained Marks : ", a)
```

```
    print ("Percentage : ", c)
    if c >= 90:
        print ("Grade A")
    elif c >= 85:
        print ("Grade A-")
```

-
-
-

```
    i += 1
```

Result:

```
Enter total number of students : 3
Enter Student Name : Umar
Enter Student Roll no. : 43
Enter Obtained Marks : 250
Name :  Umar
Roll NO. :  43
Obtained Marks :  250
Percentage :  83.33333333333333
Grade B+
Enter Student Name : Hadi
Enter Student Roll no. : 36
Enter Obtained Marks : 200
Name :  Hadi
Roll NO. :  36
Obtained Marks :  200
Percentage :  66.66666666666667
Grade C+
```

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Mannual Report	**Dated:** 28-06-2026
Semester: 1st Semester	**Session:** 2026 Spring
Lab/Project/Assignment #: 1	**CLOs to be covered:** CLO 1
Lab Title: Nested Loop	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No. 4 :-

Lab Objectives:

To understand and implement Nested Loops in Python, and to practice using a loop inside another loop to solve problems involving multi-dimensional data and patterns.

Equipment Required:

- A computer with Python installed
- Any Python IDE (e.g., VS Code, PyCharm, or IDLE)

Theory:

A **Nested Loop** in Python is a loop that exists inside the body of another loop. The outer loop controls the number of iterations, while the inner loop executes completely for each iteration of the outer loop. Nested loops are commonly used for working with **matrices, patterns, and multi-dimensional data structures**. Both for and while loops can be nested within each other.

Lab Work:

Task# 1:

```
for i in range (1,5):
    for j in range (i):
        print ("*", end = " ")
    print ()
```

Result:

Task # 2:

```
for i in range(1,5):
        for j in range(4-i):
                print ('1 ',end=("\t "))
        for i in range(2*i-1):
                print ('*',end=("\t "))
    print ( )
```

Result:

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Mannual Report	**Dated:** 28-06-2026
Semester: 1st Semester	**Session:** 2026 Spring
Lab/Project/Assignment #: 1	**CLOs to be covered:** CLO 2
Lab Title: Functions	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No. 5 :-

Lab Objectives:

To understand and implement Functions in Python, and to practice defining, calling, and returning values from functions to write modular and reusable code.

Equipment Required:

- A computer with Python installed
- Any Python IDE (e.g., VS Code, PyCharm, or IDLE)

Theory:

A **Function** in Python is a reusable block of code that performs a specific task. Functions are defined using the def keyword followed by a function name and parentheses. They can accept **parameters** as input and return values using the return statement. Functions help in making code more **organized, readable, and reusable**. Python also has built-in functions like print(), input(), and len(), as well as user-defined functions created by the programmer.

Lab Work:

Task # 1:

```
def info():
        return (print("Umar, Hadi","\nRoll NO. : 43,36","\nBank Pin :
                2231,2441"))
print (info())
```

Result:

```
Umar, Hadi
Roll NO. : 43,36
Bank Pin : 2231,2441
None
None
```

Task # 2:

```
def sum(a,b):
        return a+b
print (sum(3,4))
```

Result:

```
7
```

Task # 3:

```
def sum(a,b):
        return a+b
print (sum(c,d))
```

Result:

4

Task # 4:

```
def person(name,roll):
        print ("How are you ? ",name,"Roll no. : ",roll)
person ("Umar","43")
person ("Hadi","36")
person ("Mahar Saab","27")
person ("Ahmad","37")
```

Result:

```
How are you ?  Umar Roll no. :  43
How are you ?  Hadi Roll no. :  36
How are you ?  Mahar Saab Roll no. :  27
How are you ?  Ahmad Roll no. :  37
```

University Of Engineering and Technology, Lahore **Computer Engineering Department**

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Mannual Report	**Dated:** 28-06-2026
Semester: 1st Semester	**Session:** 2026 Spring
Lab/Project/Assignment #: 1	**CLOs to be covered:** CLO 2
Lab Title: Local & Global Variables	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.

Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No. 6 :-

Lab Objectives:

To understand the concept of Local and Global Variables in Python, and to practice the scope and accessibility of variables within and outside functions.

Equipment Required:

- A computer with Python installed
- Any Python IDE (e.g., VS Code, PyCharm, or IDLE)

Theory:

A **Local Variable** is a variable declared inside a function and can only be accessed within that function. A **Global Variable** is declared outside any function and can be accessed throughout the entire program, including inside functions. To modify a global variable inside a function, the global keyword must be used. Understanding variable scope is important to avoid naming conflicts and to control how data is accessed and modified across a program.

Lab Work:

Task # 1:

```
print(x+y)

def welcome():

  global x

  x = 45

  print (x)

welcome ()
```

Result:

```
25Hadi
45
```

Task # 2:

```
print (x+y)
def welcome():
    x = 45
    print (globals()["x"])
    print (x)
welcome ()
```

Result:

```
25Hadi
25
45
```

University Of Engineering and Technology, Lahore **Computer Engineering Department**

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Mannual Report	**Dated:** 28-06-2026
Semester: 1st Semester	**Session:** 2026 Spring
Lab/Project/Assignment #: 1	**CLOs to be covered:** CLO 3
Lab Title: List & Tuples	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No. 7 :-

Lab Objectives:

To understand and implement Lists and Tuples in Python, and to practice creating, accessing, modifying, and performing operations on these data structures.

Equipment Required:

- A computer with Python installed
- Any Python IDE (e.g., VS Code, PyCharm, or IDLE)

Theory:

A **List** is a mutable, ordered collection of items in Python that can be modified after creation (items can be added, removed, or changed), and is defined using square brackets []. A **Tuple** is an immutable, ordered collection of items that cannot be changed once created, and is defined using parentheses (). Both lists and tuples can store multiple data types and allow indexing and slicing, but tuples are generally faster and used for fixed data, while lists are preferred for data that may change.

Lab Work:

Task # 1:

```
fruits = ["Apple", "Banana", "Mango", "Orange"]
print (fruits)
print (fruits[2])
```

Result:

```
['Apple', 'Banana', 'Mango', 'Orange']
Mango
```

Task # 2:

```
numbers = [10, 20, 30, 40]
numbers.append(50)
numbers[1] = 25
print (numbers)
```

Result:

```
[10, 25, 30, 40, 50]
```

Task # 3:

```
colors = ("Red", "Green", "Blue")
print (colors)
print (colors[0])
```

Result:

```
('Red', 'Green', 'Blue')
Red
```

Task # 4:

```
t1 = (1, 2, 3)
t2 = (4, 5)
t3 = t1 + t2
print (t3)
print (len(t3))
```

Result:

```
(1, 2, 3, 4, 5)
5
```

Assignment Type: Mannual Report	**Dated:** 28-06-2026
Semester: 1st Semester	**Session:** 2026 Spring
Lab/Project/Assignment #: 1	**CLOs to be covered:** CLO 3
Lab Title: Sets & Dictionaries	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No. 8 :-

Lab Objectives:

To understand and implement Sets and Dictionaries in Python, and to practice creating, accessing, modifying, and performing operations on these data structures.

Equipment Required:

- A computer with Python installed
- Any Python IDE (e.g., VS Code, PyCharm, or IDLE)

Theory:

A **Set** is an unordered, mutable collection of unique items in Python, defined using curly braces {}, which automatically removes duplicate values and supports operations like union, intersection, and difference. A **Dictionary** is an unordered, mutable collection of **key-value pairs**, defined using curly braces {} with keys and

values separated by a colon :. Dictionaries allow fast lookup of values using their corresponding keys, and both data structures are widely used for organizing and managing data efficiently.

Lab Work:

Task # 1:

```
s =  {1,2,"hadi shahid",3,}
print (s)
```

Result:

```
{1, 2, 3, 'hadi shahid'}
```

Task # 2:

```
s = set([1,2,3,4])
print (s)
print (type(s))
```

Result:

```
{1, 2, 3, 4}
<class 'set'>
```

Task # 3:

```
fr = {"apple":"saeeb","Mango":"Aam","Watermelon":"Turbooz"}
print (fr["Watermelon"])
```

Result:

```
Turbooz
```

Task # 4:

```
s = {"Ce","Ce","Math","Cys"}
for i in (s):
    if i == "Ce":
        print (i)
```

Result:

```
Ce
```

Task # 5:

```
s.discard("Cys")
print(s)
s.discard("Ce")
print(s)
```

Result:

```
{'Ce', 'Math'}
{'Math'}
```

Task # 6:

```
s = {"Ce","Ce","Math","Cys"}
s.clear()
print (type(s))
```

Result:

```
<class 'set'>
```

Task # 7:

```
print (set.union(s1,s2))
s1.update(s2)
print(s1)
```

Result:

```
{1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
{3, 4, 5}
```

University Of Engineering and Technology, Lahore **Computer Engineering Department**

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Mannual Report	**Dated:** 28-06-2026

Semester: 1st Semester	**Session:** 2026 Spring
Lab/Project/Assignment #: 1	**CLOs to be covered:** CLO
Lab Title: Viva	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No. 9 :-

Viva of the CEA Task given after Mids.

□□□□
□

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Mannual Report	**Dated:** 28-06-2026
Semester: 1st Semester	**Session:** 2026 Spring

Lab/Project/Assignment #: 1	**CLOs to be covered:** CLO 2
Lab Title: Recursion	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No. 10 :-

Lab Objectives:

To understand and implement Recursion in Python, and to practice writing recursive functions that solve problems by calling themselves with smaller sub-problems.

Equipment Required:

- A computer with Python installed
- Any Python IDE (e.g., VS Code, PyCharm, or IDLE)

Theory:

Recursion is a programming technique in which a function calls itself directly or indirectly to solve a problem. A recursive function must have a **base case** that stops the recursion and a **recursive case** that breaks the problem into smaller sub-problems. Recursion is commonly used for problems like factorial calculation, Fibonacci series, and tree traversal. While recursion can simplify code, excessive recursive calls without a proper base case can lead to a **stack overflow error**.

Lab Work:

Task # 1:

```
def fac(n):
    if (n == 0 or n==1):
        return 1
    else:
        return n* fac(n-1)
print (fac(6))
```

Result:

```
720
```

Task # 2:

```
def fiba(n):
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fiba(n-1) + fiba(n-2)
for i in range(num):
        print (fiba(i))
```

Result:

```
Enter the number of which you want to calculate the Fibannaci Series : 7
0
1
1
2
3
5
8
```

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Mannual Report	**Dated:** 28-06-2026
Semester: 1st Semester	**Session:** 2026 Spring
Lab/Project/Assignment #: 1	**CLOs to be covered:** CLO 4
Lab Title: Exception Handling	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No. 11 :-

Lab Objectives:

To understand and implement Exception Handling in Python, and to practice detecting, catching, and managing runtime errors to make programs more robust and reliable.

Equipment Required:

- A computer with Python installed
- Any Python IDE (e.g., VS Code, PyCharm, or IDLE)

Theory:

Exception Handling is a mechanism in Python used to handle runtime errors and prevent program crashes. It is implemented using the try, except, else, and finally blocks. Code that might raise an error is placed inside the try block, while the except block catches and handles the error if one occurs. The else block executes if no exception occurs, and the finally block always executes regardless of whether an exception occurred or not. Common exceptions include ZeroDivisionError, ValueError, and TypeError.

Lab Work:

Task # 1:

```
try:
    name = int(input("Enter any Number : "))
    print (name)
except Exception as e:
    print (e)
print ("Hello World")
```

Result:

When User Enters a **Valid** Number,e,g;

```
Enter any Number : 25
25
Hello World
```

When User Enters an **In-valid** Number,e,g;

```
Enter any Number : abc
invalid literal for int() with base 10: 'abc'
Hello World
```

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Mannual Report	**Dated:** 28-06-2026
Semester: 1st Semester	**Session:** 2026 Spring
Lab/Project/Assignment #: 1	**CLOs to be covered:** CLO 2
Lab Title: Lambda Functions	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No. 12 :-

Lab Objectives:

To understand and implement Lambda Functions in Python, and to practice writing concise, anonymous functions for short, single-expression tasks.

Equipment Required:

- A computer with Python installed
- Any Python IDE (e.g., VS Code, PyCharm, or IDLE)

Theory:

A **Lambda Function** is a small, anonymous function in Python defined using the lambda keyword instead of def. It can take any number of arguments but can only contain a single expression, which is automatically returned. Lambda functions are commonly used for short, throwaway operations and are often combined with built-in functions like map(), filter(), and sorted() to perform quick transformations on data without formally defining a named function.

Lab Work:

Task # 1:

```
a = lambda x: x*x

print (a(3))
```

Result:

```
9
```

Task # 2:

```
a = lambda x,y,z: x+y+z
print (a(2,3,2))
```

Result:

```
7
```

Task # 3:

```
b = [1,2,3,4]

a = list(map(lambda x : x**2,b))

print (a)
```

Result:

```
[1, 4, 9, 16]
```

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Mannual Report	**Dated:** 28-06-2026
Semester: 1st Semester	**Session:** 2026 Spring
Lab/Project/Assignment #: 1	**CLOs to be covered:** CLO 2
Lab Title: Map, Filter & Reduce	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No. 13 :-

Lab Objectives:

To understand and implement the Map, Filter, and Reduce functions in Python, and to practice applying functional programming techniques for transforming and processing data efficiently.

Equipment Required:

- A computer with Python installed
- Any Python IDE (e.g., VS Code, PyCharm, or IDLE)

Theory:

The **map()** function applies a given function to each item of an iterable and returns a new iterable with the transformed values. The **filter()** function applies a condition to each item of an iterable and returns only the items that satisfy that condition. The **reduce()** function, available in the functools module, applies a function cumulatively to the items of an iterable, reducing it to a single value. These functions are commonly used with **lambda functions** to write concise and efficient code for data processing without explicit loops.

Lab Work:

Task # 1:

```
b = [1,2,3,4]

a = list(map(lambda x : x**2,b))

print (a)
```

Result:

```
[1, 4, 9, 16]
```

Task # 2:

```
nums = [1, 2, 3, 4, 5, 6]

result = list(filter(lambda x: x % 2 == 0, nums))

print (result)
```

Result:

```
[2, 4, 6]
```

Task # 3:

```
from functools import reduce

nums = [1, 2, 3, 4]

result = reduce(lambda x, y: x + y, nums)
```

```
print (result)
```

Result:

```
10
```

University Of Engineering and Technology, Lahore Computer Engineering Department	
Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Mannual Report	**Dated:** 28-06-2026
Semester: 1st Semester	**Session:** 2026 Spring
Lab/Project/Assignment #: 1	**CLOs to be covered:** CLO 2
Lab Title: OS Library, if __name__ == "__main__", is vs ==	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No. 14 :-

Lab Objectives:

To understand and implement the OS Library in Python, the use of if __name__ == "__main__":, and the difference between the is and == operators.

Equipment Required:

- A computer with Python installed
- Any Python IDE (e.g., VS Code, PyCharm, or IDLE)

Theory:

The **OS Library** in Python is a built-in module that provides functions for interacting with the operating system, such as handling files, directories, and environment variables. The statement if __name__ == "__main__": is used to check whether a Python script is being run directly or being imported as a module into another script, ensuring certain code only executes when the file is run directly. The == **operator** compares the **values** of two objects, while the is **operator** compares whether two variables refer to the **same memory location (identity)**, which is an important distinction especially when working with mutable objects like lists and dictionaries.

Lab Work:

Task # 1:

```
import os

os.mkdir("Loop")
```

Result:

This code creates a new folder named **"Loop"** in the current working directory. There's no printed output — but if you check the directory, you'll see the new folder.

If the folder already exists, it will raise an error:

```
FileExistsError: [WinError 183] Cannot create a file when that file already
exists: 'Loop'
```

Task # 2:

```
import os

for file in os.listdir("."):

    if file.endswith(".py"):

        size  = os.path.getsize(file)

        print (f"{file} → {size} bytes")
```

Result:

This code lists all **.py** files in the current directory along with their file sizes. The exact output depends on what **.py** files exist in your folder, but it would look something like:

```
script1.py → 245 bytes
lab5.py → 612 bytes
test.py → 89 bytes
```

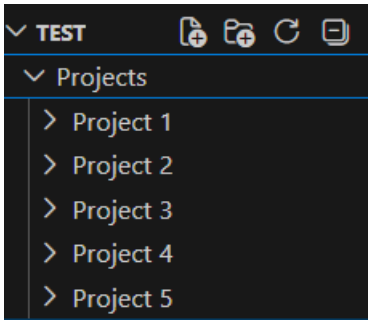
If there are **no** .py files in the current directory, there will be **no output** at all.

Task # 3:

```
import os

os.mkdir("Projects")

for i in range(1, 6):

        os.makedirs(f"Projects/Project {i}")

        open(f"Projects/Project {i}/main.py", "w")
```

Result:

This code creates a folder structure with no printed output, but it builds the following on disk:



Each main.py file will be created **empty** and if the file already **exists** then it will raise an **error.**

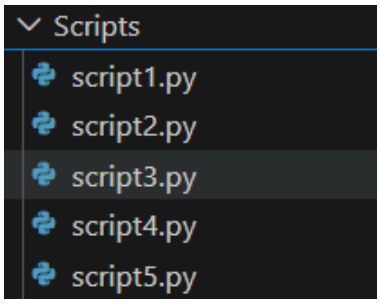
```
/Untitled-1.py
Traceback (most recent call last):
  File "c:\Users\mo_mo\Downloads\Test\Untitled-1.py", line 2, in <module>
    os.mkdir("Projects")
FileExistsError: [WinError 183] Cannot create a file when that file already exists: 'Projects'
PS C:\Users\mo_mo\Downloads\Test>
```

Task # 4:

```
import os

os.mkdir("Scripts")

for i in range(1, 6):

        open(f"Scripts/script{i}.py", "w")
```

Result:

This code creates no printed output, but it builds the following on disk:



Each script1.py through script5.py will be created as **empty files** and if the file already **exists** then it will raise an **error.**

```
/Untitled-1.py
Traceback (most recent call last):
  File "c:\Users\mo_mo\Downloads\Test\Untitled-1.py", line 2, in <module>
    os.mkdir("Scripts")
FileExistsError: [WinError 183] Cannot create a file when that file already exists: 'Scripts'
PS C:\Users\mo_mo\Downloads\Test>
```

University Of Engineering and Technology, Lahore Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Mannual Report	**Dated:** 28-06-2026
Semester: 1st Semester	**Session:** 2026 Spring
Lab/Project/Assignment #: 1	**CLOs to be covered:** CLO 3
Lab Title: Gui using Qt Designer	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No. 15 :-

Lab Objectives:

To understand and implement GUI (Graphical User Interface) development using Qt Designer, and to practice designing, building, and connecting interactive interfaces with Python through PyQt5.

Equipment Required:

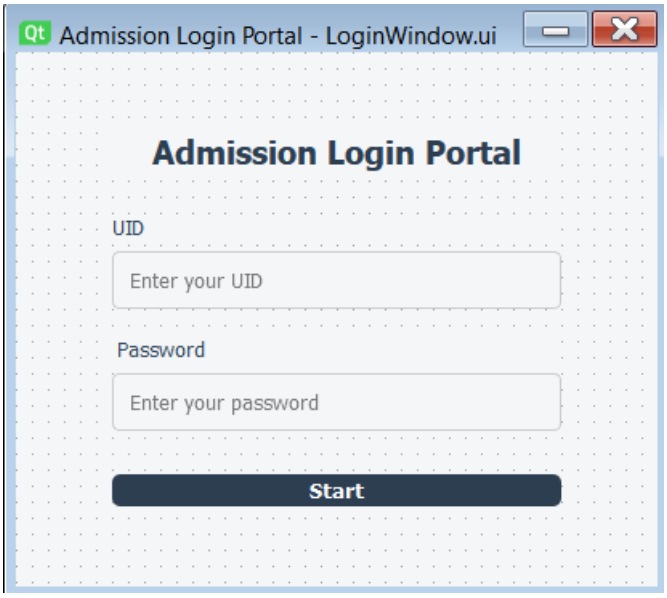
- A computer with Python installed
- Any Python IDE (e.g., VS Code, PyCharm)
- PyQt5 and Qt Designer installed

Theory:

Qt Designer is a visual tool used to design Graphical User Interfaces (GUIs) by dragging and dropping widgets such as buttons, labels, text boxes, and forms onto a canvas, without writing manual layout code. The designed interface is saved as a .ui file, which can then be converted into Python code or loaded directly using **PyQt5**. PyQt5 is a set of Python bindings for the Qt application framework that allows developers to create interactive desktop applications. Using Qt Designer along with PyQt5 significantly speeds up GUI development by separating interface design from application logic.

Lab Work:

Front-End:



Back-End:

```
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
 <class>LoginWindow</class>
 <widget class="QWidget" name="LoginWindow">
  <property name="geometry">
   <rect>
```

```
  <x>0</x>
  <y>0</y>
  <width>400</width>
  <height>320</height>
 </rect>
</property>
<property name="windowTitle">
 <string>Admission Login Portal</string>
</property>
<property name="styleSheet">
 <string notr="true">background-color: #f4f6f8;</string>
</property>
<layout class="QVBoxLayout" name="mainLayout">
 <property name="leftMargin">
  <number>60</number>
 </property>
 <property name="rightMargin">
  <number>60</number>
 </property>
 <property name="topMargin">
  <number>50</number>
 </property>
 <property name="bottomMargin">
  <number>50</number>
 </property>
 <item>
  <widget class="QLabel" name="titleLabel">
   <property name="styleSheet">
    <string notr="true">font-size: 20px; font-weight: bold; color:
#2c3e50; margin-bottom: 20px;</string>
   </property>
   <property name="text">
    <string>Admission Login Portal</string>
   </property>
   <property name="alignment">
    <set>Qt::AlignCenter</set>
   </property>
  </widget>
 </item>
 <item>
  <widget class="QLabel" name="uidLabel">
   <property name="styleSheet">
    <string notr="true">font-size: 13px; color: #34495e;</string>
   </property>
   <property name="text">
    <string>UID</string>
   </property>
  </widget>
 </item>
 <item>
```

```
            <widget class="QLineEdit" name="uidLineEdit">
             <property name="styleSheet">
              <string notr="true">padding: 8px; border: 1px solid #ccc; border-
radius: 4px;</string>
             </property>
             <property name="placeholderText">
              <string>Enter your UID</string>
             </property>
            </widget>
           </item>
           <item>
            <widget class="QLabel" name="passwordLabel">
             <property name="styleSheet">
              <string notr="true">font-size: 13px; color: #34495e; margin-top:
10px;</string>
             </property>
             <property name="text">
              <string>Password</string>
             </property>
            </widget>
           </item>
           <item>
            <widget class="QLineEdit" name="passwordLineEdit">
             <property name="styleSheet">
              <string notr="true">padding: 8px; border: 1px solid #ccc; border-
radius: 4px;</string>
             </property>
             <property name="echoMode">
              <enum>QLineEdit::Password</enum>
             </property>
             <property name="placeholderText">
              <string>Enter your password</string>
             </property>
            </widget>
           </item>
           <item>
            <widget class="QPushButton" name="startButton">
             <property name="minimumHeight">
              <number>40</number>
             </property>
             <property name="styleSheet">
              <string notr="true">background-color: #2c3e50; color: white; font-
weight: bold; border-radius: 5px; margin-top: 20px;</string>
             </property>
             <property name="text">
              <string>Start</string>
             </property>
            </widget>
           </item>
          </layout>
```

```
 </widget>
 <resources/>
 <connections/>
</ui>
```